Table of Contents

Typescript 1
Managing State Variables with useEffect and useState 2
SQLLite 7
Enabling Update and Delete 11
Working with Supabase API 15

Typescript

TypeScript is a version of JavaScript that adds static typing. Its main strengths are catching errors early and improving code clarity. It can be added gradually to existing JavaScript code.

Example 1: Creating a Typescript Project

This example contains steps for setting up a typescript React Native app. Run this command to create a basic React Native app with TypeScript using Expo:

```
npx create-expo-app myapp --template blank-typescript
cd myapp
```

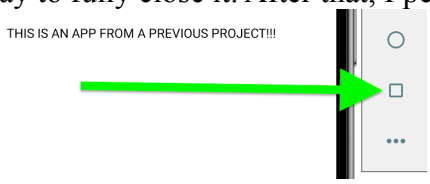
> **Mac users who may have permissions issues**
> Remember to use **sudo** if you encounter permission issues. Also, run this command at the root of your project to grant read write permission on my project folder and sub folders:
>
> ```
> sudo chown -R $USER ./
> ```

To run the app, use this command:

```
npx expo start
```

Be sure to press **a** to run the app in the Android emulator which should already be running.

> **Tip:** I often find the old app stays on the screen, so I go to the home screen and swipe it away to fully close it. After that, I perform a hard refresh to reload the app cleanly.



Managing State Variables with useEffect and useState

`useState` is used to store and update data in a component. It lets your app remember values like user input or lists. When the state changes, the screen automatically updates.

`useEffect` is used to run code when something happens in your component, like when it first loads or when state changes. It's commonly used for things like loading data, setting up a database, or running side effects.

```
useEffect(() => {
  console.log("Count changed:", count); // This runs every time the state var count
}, [count]);                            // changes.

useEffect(() => {
  console.log("App started.");          // The empty array means call once only
}, []);                                 // when the component mounts.
```

Example 2: useEffect and useState

This example creates a counter that increments it when a button is pressed.

```
import { useState, useEffect } from 'react';
import { View, Text, Button } from 'react-native';

export default function App() {
  // create a variable count and set it to 0.
  // create function 'setCount' to update the value of count.
  const [count, setCount] = useState(0);

  // Hook that runs something every time count changes.
  useEffect(() => {
    console.log("Count changed:", count);
  }, [count]);

  return (
    <View style={{ padding: 20 }}>
      <Text style={{ fontSize: 24 }}>Count: {count}</Text>

      <Button title="Increase" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

Exercise 1 (1 mark)

What line of code could be changed to initialize count to 25? Show the new line here:

```
const [count, setCount] = useState(0);
const [count, setCount] = useState(25);
```

Exercise 2 (1 mark)

What line of code would increment the state variable, count, by 2 instead of by 1 when the button is pressed?

```
<Button title="Increase" onPress={() => setCount(count + 1)} />
<Button title="Increase" onPress={() => setCount(count + 2)} />
```

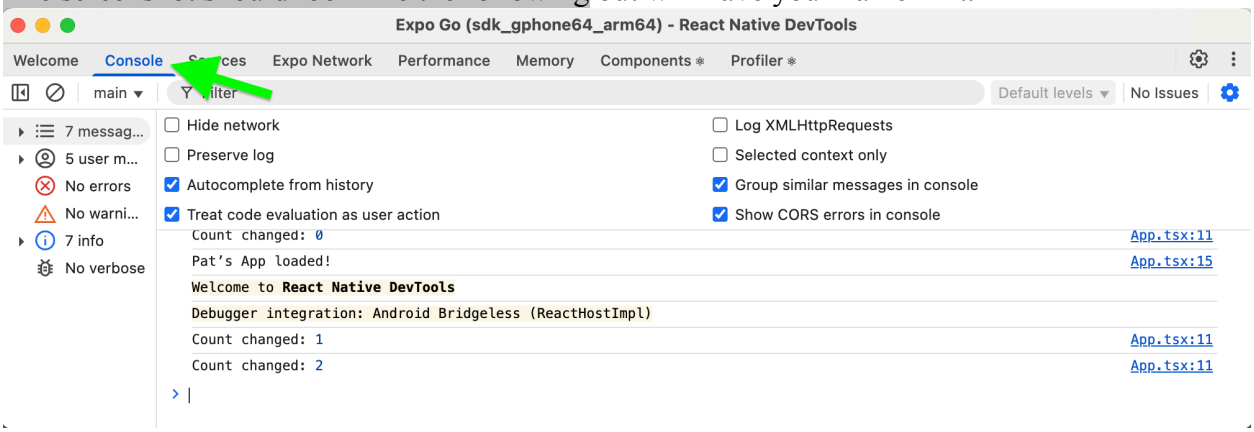
Exercise 3 (2 marks)

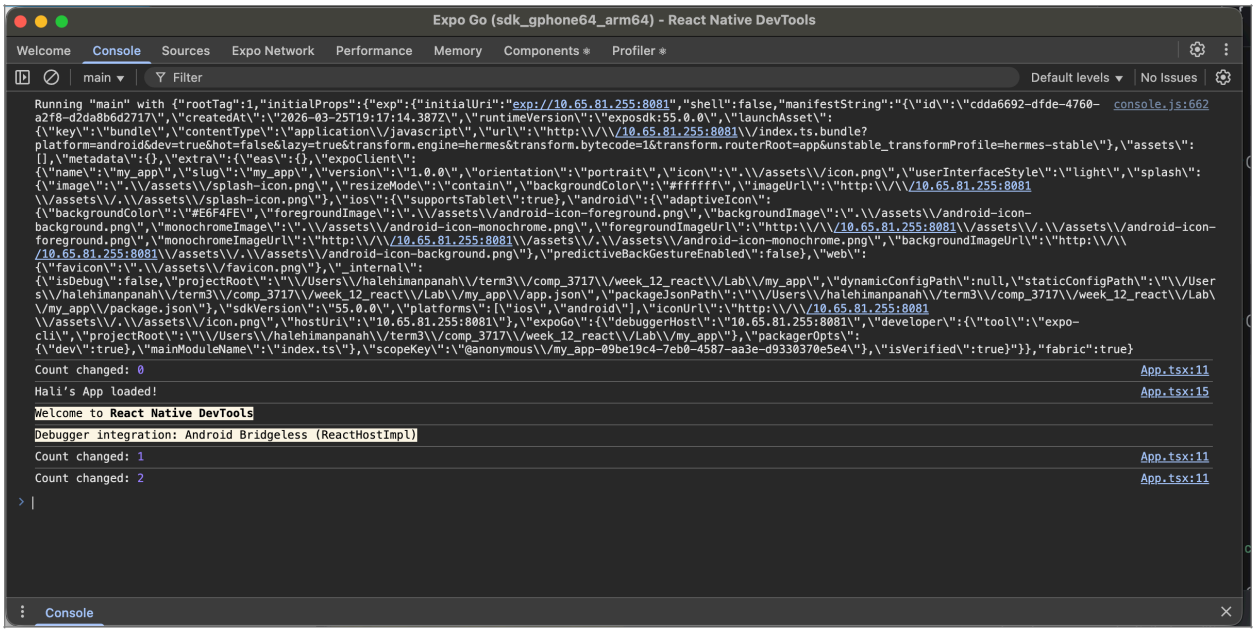
Add this code to Example 1 and include your name instead of mine.

```
useEffect(() => {
  console.log("Pat's App loaded!");
}, []);
```

After reloading the app with the code from this example, click the "Increase" button. Then show a screenshot of the debugger window which shows logging for the app loading and for the number increase. However, change the name to be your first name. To open the dev tools press **j** in the terminal where the app is launched.

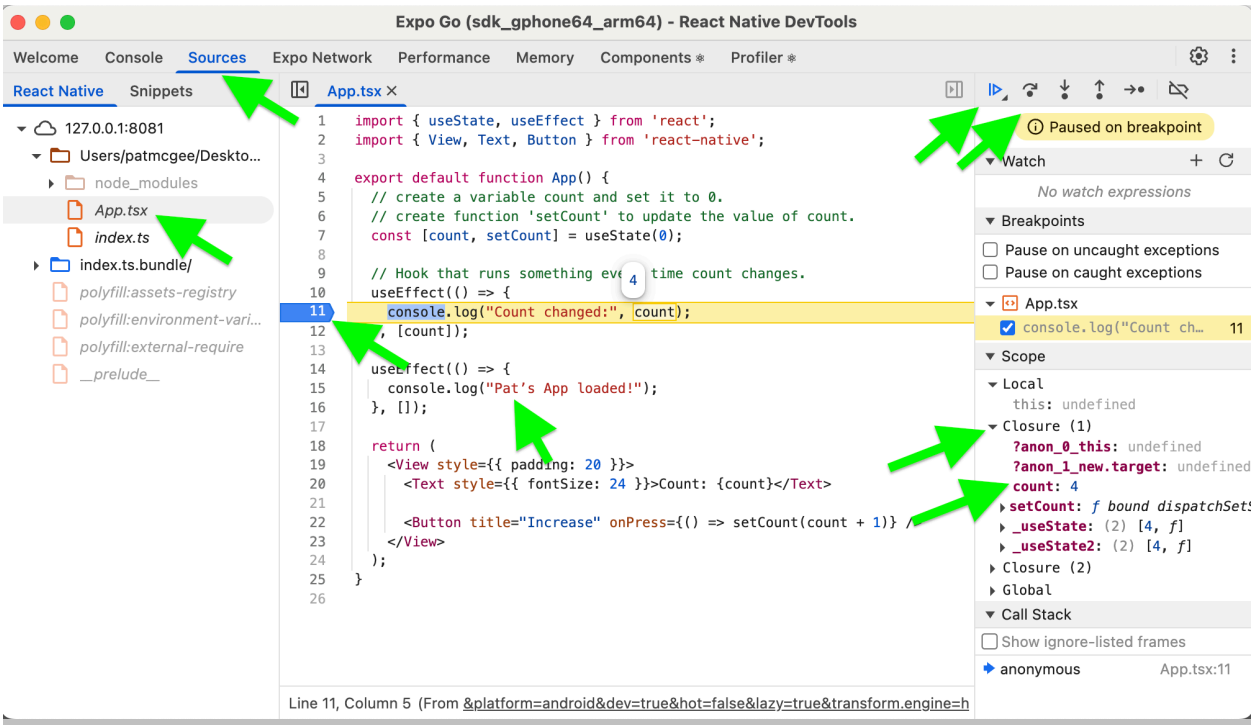
The screenshot should look like the following but will have your name in it.



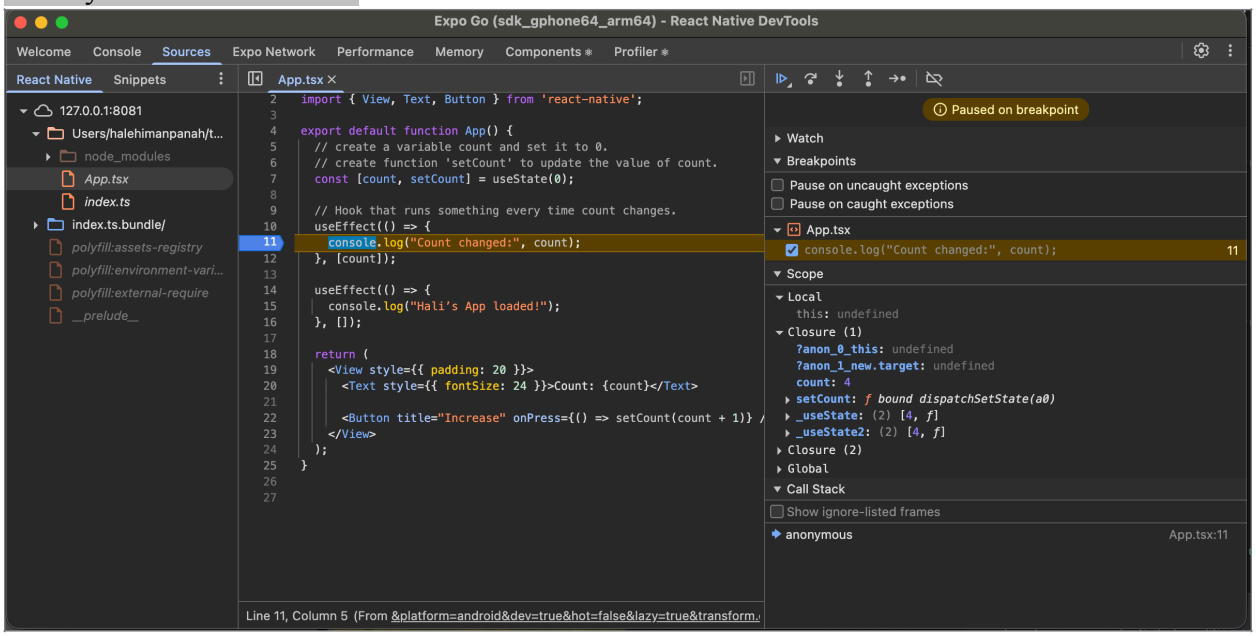


Exercise 4 (2 marks)

Set a breakpoint in the **Sources tab** in the dev tools debugger. Click on is **App.tsx** to open the main component. Halt at the console.log() instruction so you can inspect what's happening. At the top right, the debugger controls (resume and step buttons) allow you to continue execution or move through the code step-by-step. On the right-hand side, the scope panel reveals the current state of variables, showing that `count` is 4 at this moment. Show a screenshot while halted in the sources tab beside the log instruction that has your name in it while count is displayed on the right in the closure. It should look something like this but have your name in it.



Show your screenshot here:



Exercise 5 (1 mark)

Explain the difference between both *useEffect()* instructions that exist after completing Exercise 3.

.

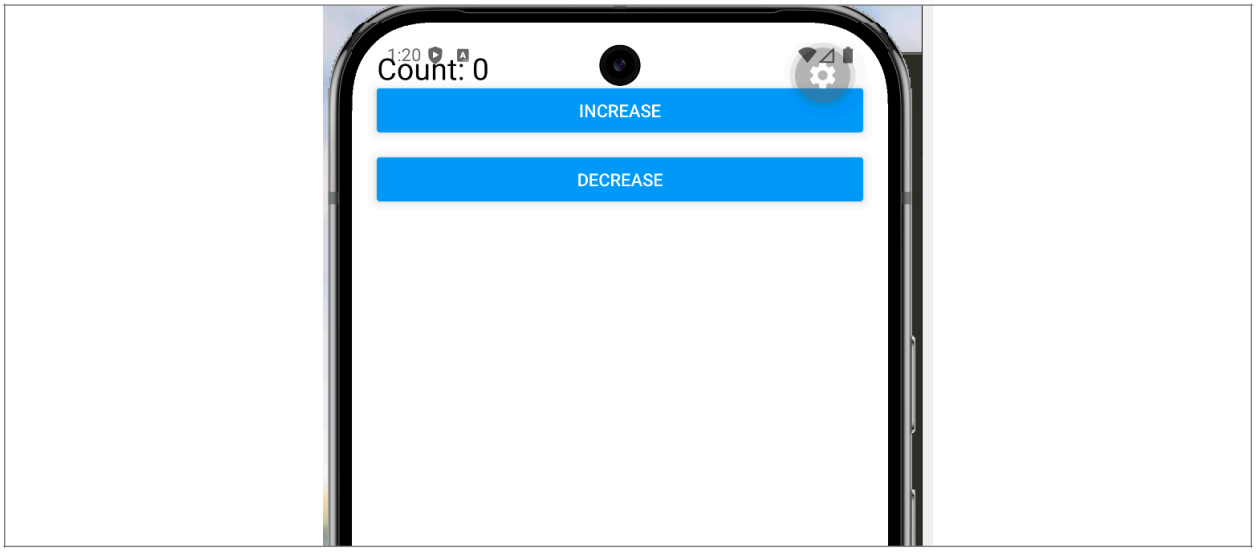
One runs repeatedly whenever count changes, One runs only once on startup

Exercise 6 (1 mark)

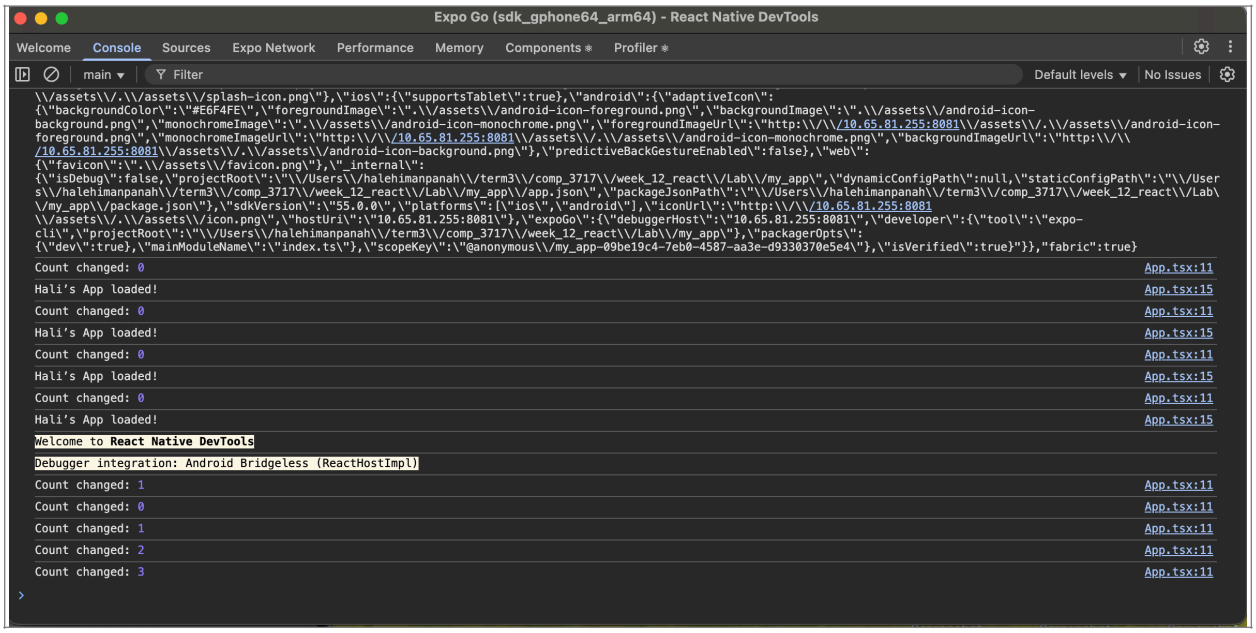
Add a second button that will decrease the count value. Show the code used here:

```
<Button title="Decrease" onPress={() => setCount(count - 1)} />
```

Show a screenshot of the new interface after the button has been added:



Show a screenshot of the logging which displays changes to count as the increase and decrease buttons are clicked:

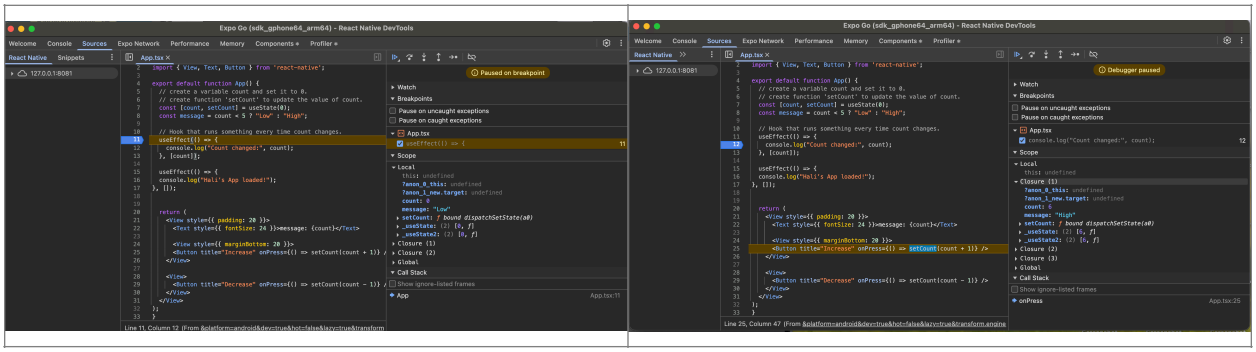


Exercise 7 (1 mark)

Add this declaration to Example 2:

```
  const message = count < 5 ? "Low" : "High";
```

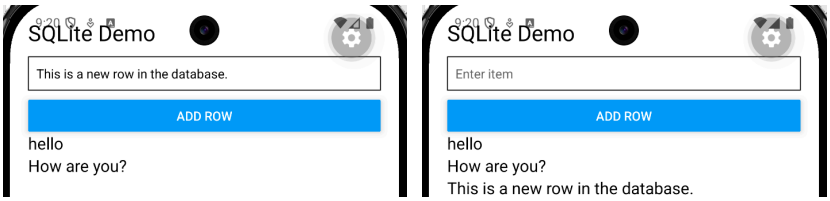
Show the message value in a Text component. Show screenshots of the screen for both settings when count is below and above 5.



SQLLite

Example 3: Using SQLite

This example sets up a basic SQLite app that has persistent storage. The app is able to add and view data.



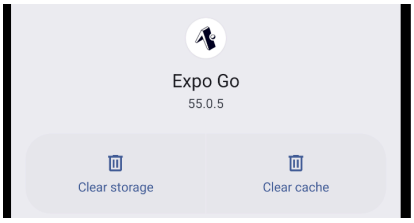
After creating an app like the one in Example 1, while the command prompt is located in the folder where the package.json file resides for your project run this command to download and install the expo-sqlite package:

```
npx expo install expo-sqlite
```

Warning: Data from the Previous App Remains

If you create a new app that uses a SQLite database you may find that data from an old project remains. This happens because the SQLite database is stored on the device, so a new app with the same name reuses it. To fix it, use a different database name or uninstall/clear the app data. Also, fully close and reload the app to avoid cached data.

To clear the data within the emulator go to **Settings | Apps | Your App | Storage | Clear Data**. Depending on your emulator version you will be presented with something like the following:

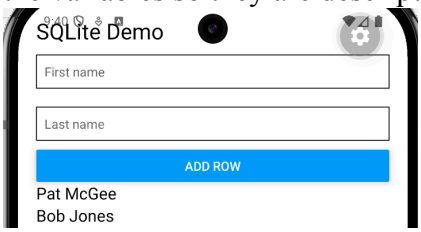


Here is the code:

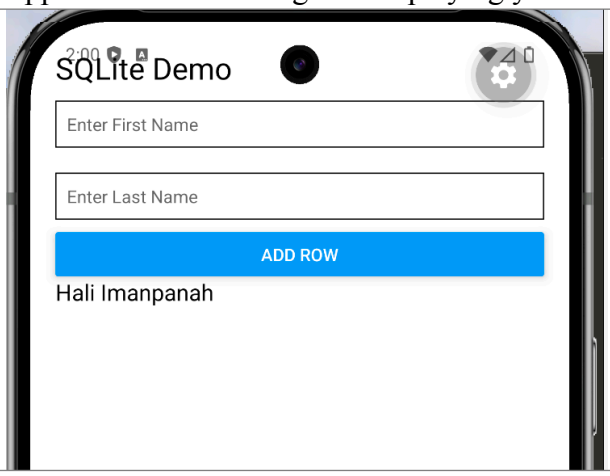
```
import { useEffect, useState } from 'react';
import { View, Text, Button, TextInput, FlatList } from 'react-native';
import * as SQLite from 'expo-sqlite';

// Open database
const db = SQLite.openDatabaseSync('mydb.db');

export default function App() {
  const [text, setText] = useState('');
  const [rows, setRows] = useState<any[]>([]);

  // Create table on start
  useEffect(() => {
    db.execSync(`
      CREATE TABLE IF NOT EXISTS items (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT
      );
    `);
    loadData();
  }, []);

  // Insert row
  const addItem = () => {
    if (!text) return;

    db.execSync(`INSERT INTO items (name) VALUES ('${text}');`);
    setText('');
    loadData();
  };

  // Load rows
  const loadData = () => {
    const result = db.getAllSync('SELECT * FROM items;');
    setRows(result);
  };

  return (
    <View style={{ padding: 20 }}>
      <Text style={{ fontSize: 24 }}>SQLite Demo</Text>

      <TextInput
        placeholder="Enter item"
        value={text}
        onChangeText={setText}
        style={{ borderWidth: 1, marginVertical: 10, padding: 8 }}
      />
```

Exercise 8 (5 marks)

Modify Example 3 to create a different database named fullnames. Define a fullname table to store first and last names. When finished the interface should look like the following. Rename the variables so they are descriptively accurate.



Show a screenshot of the app screen after adding and displaying your name:



Show your full app.tsx file contents as text (no screenshots please). Be sure that your revised code uses proper naming:

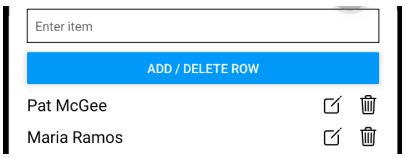
```
import { useEffect, useState } from 'react';
import { View, Text, Button, TextInput, FlatList } from 'react-native';
import * as SQLite from 'expo-sqlite';

// Open database
const db = SQLite.openDatabaseSync('fullnames.db');

export default function App() {
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
  const [fullNames, setFullNames] = useState<any[]>([]);

  // Create table on start
  useEffect(() => {
    db.execSync(`
      CREATE TABLE IF NOT EXISTS fullnames (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        first_name TEXT,
        last_name TEXT
      );
    `);
    loadData();
  }, []);

  // Insert row
  const addFullName= () => {
    if (!firstName || !lastName) return;

    db.execSync(`INSERT INTO fullnames (first_name, last_name) VALUES ('${firstName}', '${lastName}');`);
    setFirstName('');
```

```
    setLastName('');
    loadData();
  };

  // Load rows
  const loadData = () => {
    const result = db.getAllSync('SELECT * FROM fullnames;');
    setFullNames(result);
  };

  return (
    <View style={{ padding: 20 }}>
      <Text style={{ fontSize: 24 }}>SQLite Demo</Text>

      <TextInput
  placeholder="Enter First Name"
  value={firstName}
  onChangeText={setFirstName}
  style={{ borderWidth: 1, marginVertical: 10, padding: 8 }}
/>
<TextInput
  placeholder="Enter Last Name"
  value={lastName}
  onChangeText={setLastName}
  style={{ borderWidth: 1, marginVertical: 10, padding: 8 }}
/>
      <Button title="Add Row" onPress={addFullName} />

      <FlatList
        data={fullNames}
        keyExtractor={(item) => item.id.toString()}
        renderItem={({ item }) => (
          <Text style={{ fontSize: 18 }}>{item.first_name} {item.last_name}</Text>
        )}
      />
    </View>
  );
}
```

Enabling Update and Delete

Example 4: Enabling Update and Delete

This example continues from Example 3 to enable updates and deletions with existing rows of data with a little bit more formatting.



Replace App.tsx with the Example 3 code. To enable these new database operations, add this to the top of the App class.

```
  const [editId, setEditId] = useState<number | null>(null);
```

Then to enable update and insert functions replace:

```
// DELETE
const addItem = () => {
  if (!text) return;

  db.execSync(`INSERT INTO items (name) VALUES ('${text}');`);
  setText('');
  loadData();
};
```

with:

```
// ADD or UPDATE
const saveItem = () => {
  if (!text) return;

  if (editId) {
    db.execSync(`UPDATE items SET name='${text}' WHERE id=${editId};`);
    setEditId(null);
  } else {
    db.execSync(`INSERT INTO items (name) VALUES ('${text}');`);
  }
  setText('');
  loadData();
};

// DELETE
const deleteItem = (id: number) => {
  db.execSync(`DELETE FROM items WHERE id=${id};`);
  loadData();
};

// START EDIT
const startEdit = (item: any) => {
  setText(item.name);
  setEditId(item.id);
};
```

Then replace the button with a new one that is labelled differently and that calls the new method:

```
    <Button title="Add / Delete Row" onPress={saveItem} />
```

To enable a nicer looking FlatList with icons run this command:

```
npx expo install @expo/vector-icons
```

And add this import:

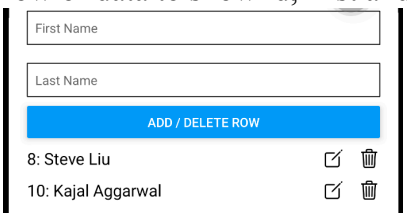
```
import { Ionicons } from '@expo/vector-icons';
```

Finally, replace the FlatList component with this version so each data row can include an edit and delete button.

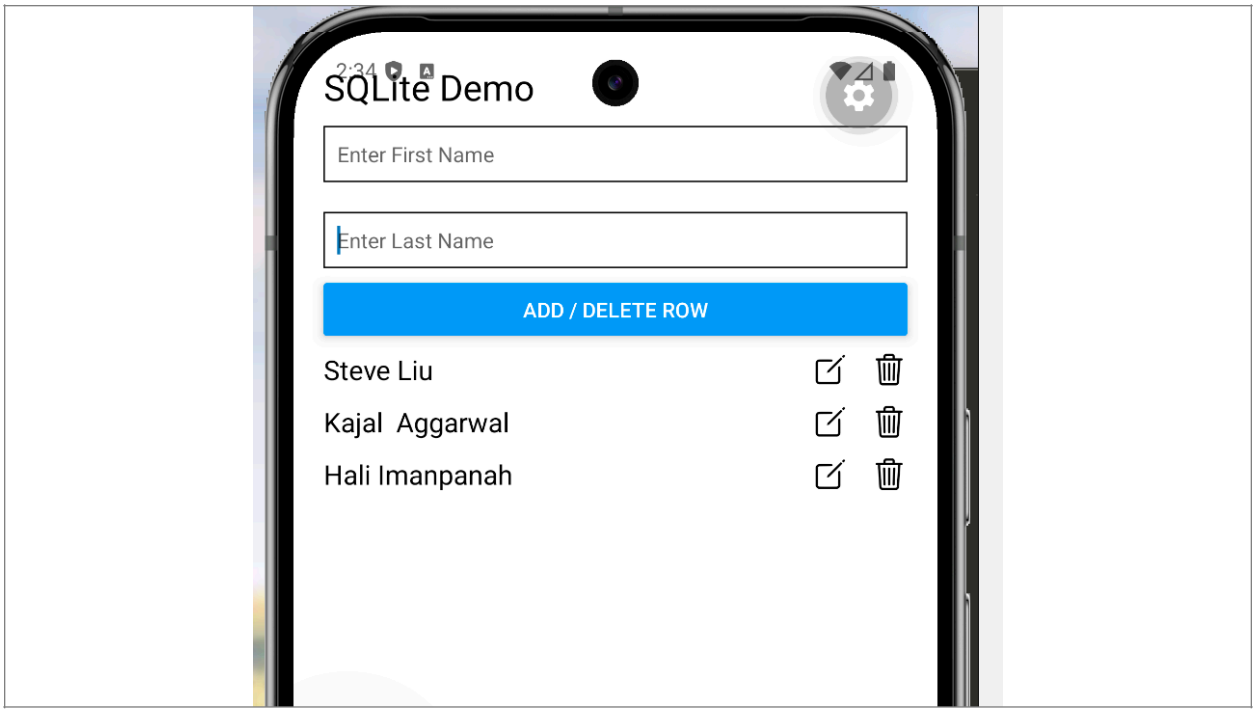
```
<FlatList
  data={rows}
  keyExtractor={(item) => item.id.toString()}
  renderItem={({ item }) => (
    <View style={{
      flexDirection: 'row',
      justifyContent: 'space-between',
      alignItems: 'center',
      marginTop: 10
    }}>

      <Text style={{ fontSize: 18 }}>{item.name}</Text>

      <View style={{ flexDirection: 'row', gap: 15 }}>
        <Ionicons
          name="create-outline"
          size={24}
          onPress={() => startEdit(item)}
        />
        <Ionicons
          name="trash-outline"
          size={24}
          onPress={() => deleteItem(item.id)}
        />
      </View>

    </View>
  )}
/>
```

Exercise 9 (5 marks)

Modify Example 4 so it has a separate text input for the first and last names. For display each row of data to show id, first and last names along with function edit and delete buttons.



Show a screenshot of the updated interface but with your name it here:



Show your full code here after making these changes:

```
import { useEffect, useState } from 'react';
import { View, Text, Button, TextInput, FlatList } from 'react-native';
import * as SQLite from 'expo-sqlite';
import { Ionicons } from '@expo/vector-icons';

// Open database
const db = SQLite.openDatabaseSync('fullnames.db');


export default function App() {
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
  const [fullNames, setFullNames] = useState<any[]>([]);
  const [editId, setEditId] = useState<number | null>(null);

  // Create table on start
  useEffect(() => {
    db.execSync(`
      CREATE TABLE IF NOT EXISTS fullnames (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        first_name TEXT,
        last_name TEXT
      );
    `);
    loadData();
  }, []);

   // ADD or UPDATE
   const saveFullName = () => {
    if (!firstName || !lastName) return;

    if (editId) {
      db.execSync(`UPDATE fullnames SET first_name='${firstName}', last_name='${lastName}' WHERE id=${editId};`);
      setEditId(null);
    } else {
      db.execSync(`INSERT INTO fullnames (first_name, last_name) VALUES ('${firstName}', '${lastName}');`);
    }
    setFirstName('');
    setLastName('');
    loadData();
  };

  // DELETE
  const deleteFullName = (id: number) => {
```

```
    db.execSync(`DELETE FROM fullnames WHERE id=${id};`);
    loadData();
  };

  // START EDIT
  const startEditFullName = (item: any) => {
    setFirstName(item.first_name);
    setLastName(item.last_name);
    setEditId(item.id);
  };

  // Load rows
  const loadData = () => {
    const result = db.getAllSync('SELECT * FROM fullnames;');
    setFullNames(result);
  };

  return (
    <View style={{ padding: 20 }}>
      <Text style={{ fontSize: 24 }}>SQLite Demo</Text>

      <TextInput
  placeholder="Enter First Name"
  value={firstName}
  onChangeText={setFirstName}
  style={{ borderWidth: 1, marginVertical: 10, padding: 8 }}
/>
<TextInput
  placeholder="Enter Last Name"
  value={lastName}
  onChangeText={setLastName}
  style={{ borderWidth: 1, marginVertical: 10, padding: 8 }}
/>
<Button title="Add / Delete Row" onPress={saveFullName} />

<FlatList
  data={fullNames}
  keyExtractor={(item) => item.id.toString()}
  renderItem={({ item }) => (
    <View style={{
      flexDirection: 'row',
      justifyContent: 'space-between',
      alignItems: 'center',
      marginTop: 10
    }}>

      <Text style={{ fontSize: 18 }}>{item.first_name} {item.last_name}</Text>

      <View style={{ flexDirection: 'row', gap: 15 }}>
        <Ionicons
          name="create-outline"
          size={24}
          onPress={() => startEditFullName (item)}
        />
        <Ionicons
          name="trash-outline"
          size={24}
          onPress={() => deleteFullName (item.id)}
        />
      </View>

    </View>
  )}
/>
    </View>
  );
}
```

Working with Supabase API

Here's a simple React Native + TypeScript + Supabase example that lets you insert, edit, update, and delete rows. This example uses a table called items with:

- id

- name

Example 5: Getting Supabase API

This example walks through the steps needed to create a Supabase database and connect to it from a React Native app through the API. You will then use the app to add data and view it on the Android screen.

First create a new project like the one in Example 1.

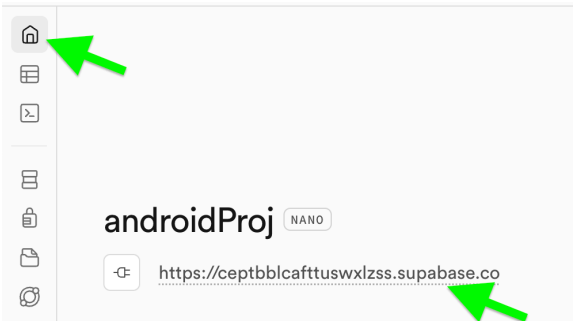
Then in the project folder where the package.json file exist, run these commands to get the required packages:

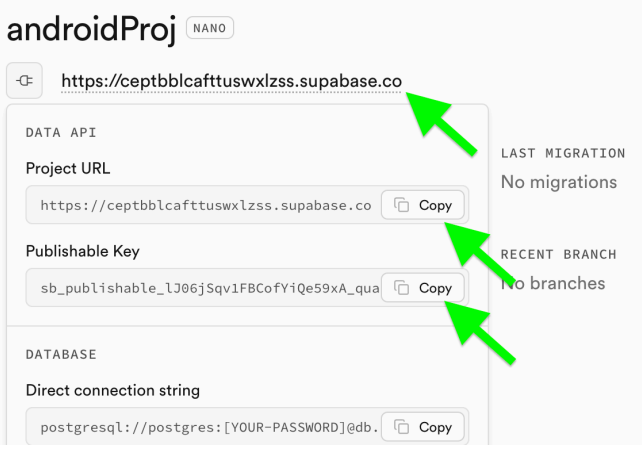
```
npx expo install react-native-url-polyfill
npm install @supabase/supabase-js
```

NextCreate **supabase.ts** with this code in it.

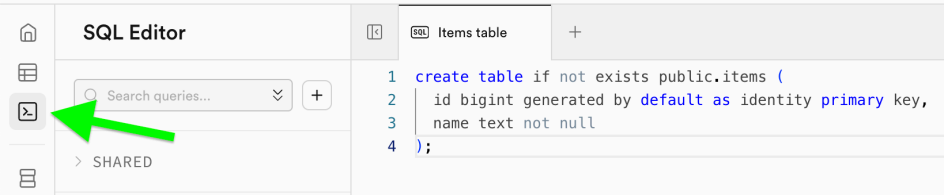
```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';
const supabaseUrl.    = 'https://ceptbblcafttuswxlzss.supabase.co';
const supabaseAnonKey = 'YOUR_ANON_KEY';
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

After creating a free Supabase account, you can get the supabaseUrl and supabaseAnonKey by hovering over the main project URL that appears at the home screen for the project. The Publishable Key is equivalent to the supabaseAnonKey.





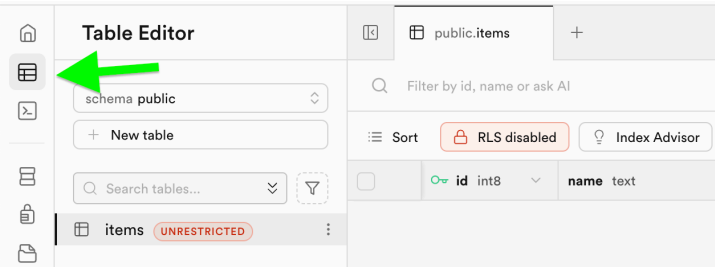
The table can be created by running the create table script in the SQL editor.



Use this in the SQL editor:

```
create table if not exists public.items (
  id bigint generated by default as identity primary key,
  name text not null
);
```

After creating the table in the **SQL Editor** the shell table is then viewable in the **Table Editor**.



App.tsx

```typescript
import { useEffect, useState } from 'react';
import { View, Text, TextInput, Button, FlatList, StyleSheet } from 'react-native';
import { supabase } from './supabase';

type Item = {
  id: number;
  name: string;
};

export default function App() {
  const [name, setName] = useState('');
  const [items, setItems] = useState<Item[]>([]);

  useEffect(() => {
    fetchItems(); // useEffect runs after the component renders
  }, []);         // The empty dependency array [] means
                  // run this effect only once (on mount)

  const fetchItems = async () => {
    const { data, error } = await supabase
      .from('items')
      .select('*')
      .order('id', { ascending: true });

    if (error) {
      console.log('Fetch error:', error.message);
      return;
    }

    setItems(data || []);
  };

  const addItem = async () => {
    if (!name.trim()) return;

    const { error } = await supabase
      .from('items')
      .insert([{ name: name.trim() }]);

    if (error) {
      console.log('Insert error:', error.message);
      return;
    }

    setName('');
    fetchItems();
  };
```

Exercise 10 (1 mark)

Starting with this code:

```
    const { error } = await supabase
      .from('items')
      .insert([{ name: name.trim() }]);
```

What happens if you remove the await? Does the program crash, or does it still run? If it still runs, what changes in the program's behavior?

without await, insert has not finished before fetchItems() is called so the new item may not show up. Errors are also silently ignored.

Exercise 11 (1 mark)

Why is fetchItems() called inside useEffect?

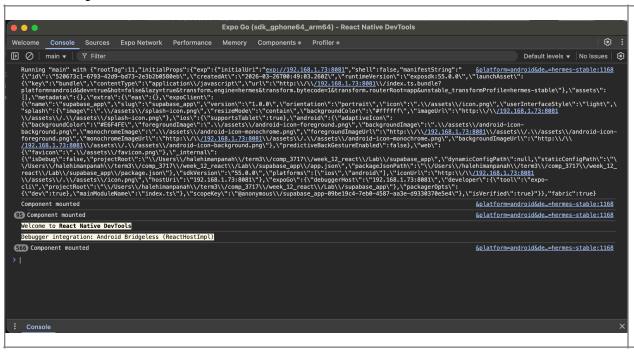
the data loads automatically when the app starts.

Exercise 12 (2 marks)

Modify the useEffect so that it no longer includes the dependency array. Run the app and capture a screenshot of the dev tools debugger output. Based on what you observe, what problem happens when the array is removed?

```
  useEffect(() => {
    fetchItems();
    console.log('Component mounted');
  }, );
```

Show your screenshot and answer here:

	useEffect runs after every render, causes an infinite loop.

Exercise 13 (1 mark0

Why do we call fetchItems() after inserting data?

await supabase.from('items').insert(...) fetchItems();

Write your answer here:

To refresh the list so that the newly inserted item shows on screen immediately after being added

Exercise 14 (1 mark)

Explain why the OR operator is used in this instruction:

```
setItems(data || []);
```

If data is null, this sign || returns an empty array instead to prevent app from being crashed

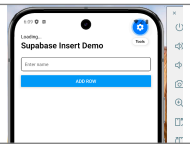
Exercise 15 (3 marks)

Modify the app so that a message appears while data is loading
(e.g., "Loading...")

Here is a code hint:

```
const [loading, setLoading] = useState(false);
```

Show your modified App.tsx with all changes. Highlight your changes. (Test the code to be sure it works).

	Message appears briefly when app starts, while data is being fetched, but disappears once the data loads. The loading state is set to true before the fetch & false after it completes

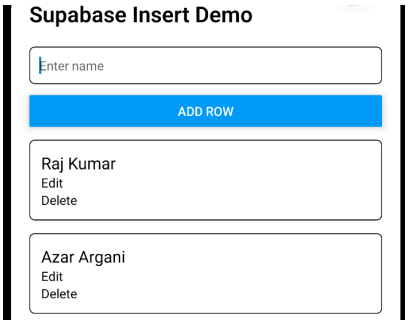
```
import { useEffect, useState } from 'react';
import { View, Text, TextInput, Button, FlatList, StyleSheet } from 'react-native';
import { supabase } from './supabase';

type Item = {
  id: number;
  name: string;
};

export default function App() {
  const [name, setName] = useState('');
  const [items, setItems] = useState<Item[]>([]);
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    fetchItems(); // useEffect runs after the component renders
  }, []);        // The empty dependency array [] means
                 // run this effect only once (on mount)

  const fetchItems = async () => {
    setLoading(true);
    const { data, error } = await supabase
      .from('items')
      .select('*')
      .order('id', { ascending: true });

    if (error) {
```

```
      console.log('Fetch error:', error.message);
      return;
    }

    setItems(data || []);
    setLoading(false);
  };

  const addItem = async () => {
    if (!name.trim()) return;

    const { error } = await supabase
      .from('items')
      .insert([{ name: name.trim() }]);

    if (error) {
      console.log('Insert error:', error.message);
      return;
    }

    setName('');
    fetchItems();
  };

  return (
    <View style={styles.container}>
      {loading && <Text>Loading...</Text>}
      <Text style={styles.title}>Supabase Insert Demo</Text>

      <TextInput
        placeholder="Enter name"
        value={name}
        onChangeText={setName}
        style={styles.input}
      />

      <Button title="Add Row" onPress={addItem} />

      <FlatList
        data={items}
        keyExtractor={(item) => item.id.toString()}
        renderItem={({ item }) => (
          <View style={styles.row}>
            <Text style={styles.name}>{item.name}</Text>
          </View>
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    paddingTop: 60,
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
    fontWeight: '600',
  },
  input: {
    borderWidth: 1,
```

```
    padding: 10,
    marginBottom: 12,
    borderRadius: 6,
  },
  row: {
    marginTop: 14,
    padding: 12,
    borderWidth: 1,
    borderRadius: 6,
  },
  name: {
    fontSize: 18,
  },
});
```

Example 6: Enabling Edit and Delete

This example builds on the previous one by adding edit and delete capability for the rows in the table.



Add this instruction to manage state for editing.

```
const [editId, setEditId] = useState<number | null>(null);
```

Then replace addItem() with

```
const saveItem = async () => {
  if (!name.trim()) return;

  if (editId !== null) {
    const { error } = await supabase
      .from('items')
      .update({ name: name.trim() })
      .eq('id', editId);

    if (error) {
      console.log('Update error:', error.message);
      return;
    }

    setEditId(null);
  } else {
    const { error } = await supabase
      .from('items')
      .insert([{ name: name.trim() }]);

    if (error) {
      console.log('Insert error:', error.message);
      return;
    }
  }

  setName('');
  fetchItems();
};
```

Next add these two functions:

```
const startEdit = (item: Item) => {
  setName(item.name);
  setEditId(item.id);
};

const deleteItem = async (id: number) => {
  await supabase.from('items').delete().eq('id', id);
  fetchItems();
};
```

Then clickable Edit and Delete triggers to the data presentation row:

```
<View style={styles.actions}>
    <TouchableOpacity onPress={() => startEdit(item)}>
    <Text style={styles.actionText}>Edit</Text>
    </TouchableOpacity>

    <TouchableOpacity onPress={() => deleteItem(item.id)}>
    <Text style={styles.actionText}>Delete</Text>
    </TouchableOpacity>
</View>
```

The TouchableOpactity component must be imported so it can be used in the previous step.

```
import { View, Text, TextInput, Button, FlatList, StyleSheet, TouchableOpacity }
from 'react-native';
```

Replace the button with this code:

```
 <Button title={editId !== null ? 'Update Row' : 'Add Row'} onPress={saveItem} />
```

You may notice that actions and actionText appear with red squiggles in the editor. This is because they are not yet defined in the StyleSheet. To fix this, add the following styles to your StyleSheet.

```
actions: {
  flexDirection: 'row',
  gap: 20,
},

actionText: {
  fontSize: 16,
},
```

Exercise 16 (1 mark)

Why is editId initialized as null?

Because no item is being edited when the app starts

Exercise 17 (1 mark)

How does the “Add Row” button change to “Update Row” when editing an item? What part of the code enables this behavior?

When startEdit() sets editId, the button title checks editId !== null and switches from Add Row to Update Row

Exercise 18 (1 mark)

Given this code snippet, how does it enable delete?

```
.eq('id', id)
```

It filters the delete operation to only remove the row where the id matches

Exercise 19 (1 mark)

What does startEdit() do?

It loads the selected item's name into the input field and sets editId so the app knows which item to update.

Exercise 20 (1 mark)

What do these two properties do in the TextEdit function?

```
value={name}
onChangeText={setName}
```

Please show your response here:

value={name} displays the current value of name in the input field. onChangeText={setName} updates name every time the user types.

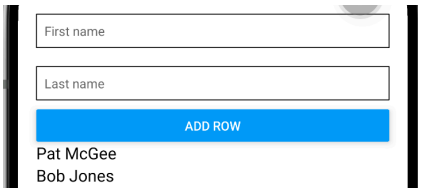
Exercise 21 (1 mark)

Why does FlatList need a key extractor?

to identify each row so it can efficiently update only the rows that changed.

Exercise 22 (5 marks)

Modify Example 6 by using a different table which includes first and last names. When finished the interface should look like the following. Rename the variables so they are descriptively accurate.



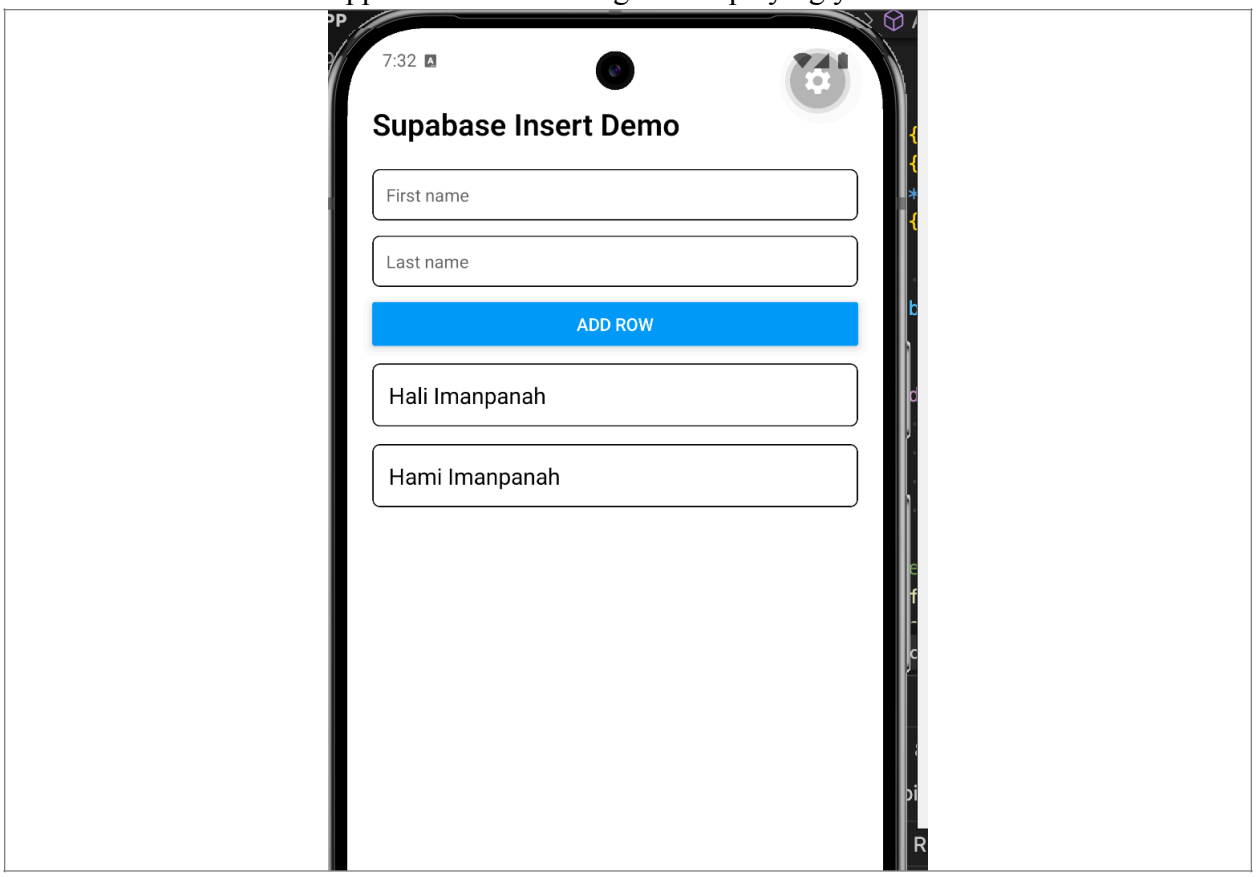
Show the new Supabase SQL Script here:

```
create table if not exists public.fullnames (
  id bigint generated by default as identity primary key,
  first_name text not null,
  last_name text not null
);
```

Show the entire App.tsx code as text here after making the changes – use proper descriptive names. No screenshots please.

```
import { useEffect, useState } from 'react';
import { supabase } from './supabase';
import { View, Text, TextInput, Button, FlatList, StyleSheet, TouchableOpacity } from 'react-
native';

type Item = {
  id: number;
  first_name: string;
  last_name: string;
};

export default function App() {
  const [firstName, setFirstName] = useState('');
  const [lastName, setLastName] = useState('');
  const [items, setItems] = useState<Item[]>([]);
  const [loading, setLoading] = useState(false);
  const [editId, setEditId] = useState<number | null>(null);

  useEffect(() => {
    fetchItems(); // useEffect runs after the component renders
  }, []);         // The empty dependency array [] means
                  // run this effect only once (on mount)

  const fetchItems = async () => {
    setLoading(true);
    const { data, error } = await supabase
      .from('fullnames')
      .select('*')
      .order('id', { ascending: true });

    if (error) {
      console.log('Fetch error:', error.message);
      return;
    }

    setItems(data || []);
    setLoading(false);
  };

  const saveItem = async () => {
    if (!firstName.trim() || !lastName.trim()) return;

    if (editId !== null) {
      const { error } = await supabase
      .from('fullnames')
        .update({ first_name: firstName.trim(), last_name: lastName.trim() })
        .eq('id', editId);

      if (error) {
        console.log('Update error:', error.message);
        return;
      }

      setEditId(null);
    } else {
      const { error } = await supabase
      .from('fullnames')
        .insert([{ first_name: firstName.trim(), last_name: lastName.trim() }]);

      if (error) {
        console.log('Insert error:', error.message);
        return;
      }
    }

    setFirstName('');
setLastName('');
    fetchItems();
  };

  const startEdit = (item: Item) => {
    setFirstName(item.first_name);
```

```
setLastName(item.last_name);
    setEditId(item.id);
  };

  const deleteItem = async (id: number) => {
    await supabase.from('fullnames').delete().eq('id', id);
    fetchItems();
  };
  return (
    <View style={styles.container}>
      {loading && <Text>Loading...</Text>}
      <Text style={styles.title}>Supabase Insert Demo</Text>

      <TextInput
  placeholder="First name"
  value={firstName}
  onChangeText={setFirstName}
  style={styles.input}
/>
<TextInput
  placeholder="Last name"
  value={lastName}
  onChangeText={setLastName}
  style={styles.input}
/>

<Button title={editId !== null ? 'Update Row' : 'Add Row'} onPress={saveItem} />

<FlatList
  data={items}
  keyExtractor={(item) => item.id.toString()}
  renderItem={({ item }) => (
    <View style={styles.row}>
      <Text style={styles.name}>{item.first_name} {item.last_name}</Text>
    </View>
  )}
/>
    </View>
  );
}


const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    paddingTop: 60,
  },
  title: {
    fontSize: 24,
    marginBottom: 20,
    fontWeight: '600',
  },
  input: {
    borderWidth: 1,
    padding: 10,
    marginBottom: 12,
    borderRadius: 6,
  },
  row: {
    marginTop: 14,
    padding: 12,
    borderWidth: 1,
    borderRadius: 6,
  },
  name: {
    fontSize: 18,
  },
  actions: {
    flexDirection: 'column',
    gap: 2,
  },
```

```
  actionText: {
    fontSize: 16,
  },
});
```

Show a screenshot of the app screen after adding and displaying your name: